## Decision Trees

**Libraries Used:**

- numpy
- pandas
- matplotlib.pyplot
- seaborn
- sklearn.tree
- sklearn.preprocessing
- sklearn.model_selection
- sklearn.base
- sklearn.metrics
- imblearn.over_sampling

### Data Preprocessing

Datatype conversion, Imputation for missing values, dropping irrelevant columns and encoding categorical features were done. These steps are explained under the preprocessing heading of neural networks.

**1. Handling Imbalance**

Several Imbalance Handling techniques were inspected, SMOTE outperformed other techniques. So SMOTE was applied to training data only. Also, decision trees split based on impurity reduction; if minority samples are few, weighted impurities often still favor majority class splits therefore class weights were not applied.

**2. Feature Reduction**

The initial model was trained using all the features to observe feature importance. After in the next model the least contributing features were removed and the model was evaluated again. The second model's F1 score dropped slightly but the recall increased enabling to miss only 99 churners.

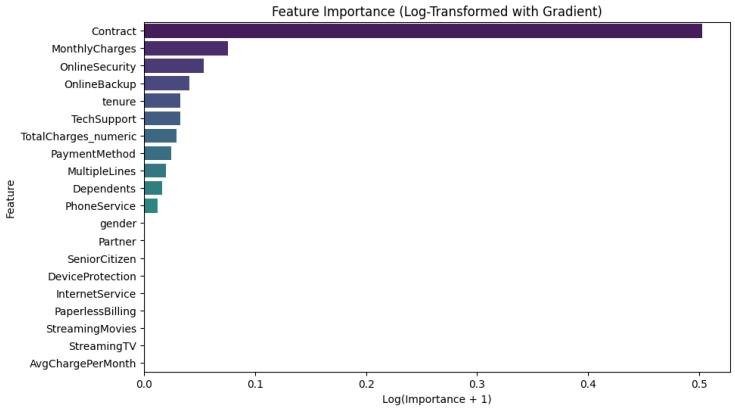



*Figure 1 - Feature Importance (before feature reduction)*

***Note:*** *The refined dataset after feature reduction was used for the final model training and evaluation. Which will be discussed below.*

## Model Implementation

The Decision Tree Classifier was initialized using the basic hyperparameters: max_depth=10, min_samples_split=15, min_samples_leaf=15, criterion='gini', random_state=42.

## Model Training

**Hyperparameter Tuning Using GridSearchCV**

- The Decision Tree model has multiple hyperparameters that strongly influence its predictive performance.
- Hyperparameter tuning helps identify the combination that maximizes the F1-score, which is crucial for imbalanced datasets like churn prediction.

**Methodology**

- A parameter grid was defined to explore different values for the key hyperparameters.
- Stratified 5-Fold Cross-Validation was used to verify that each fold maintains the original distribution of churned versus retained customers.
- GridSearchCV tests all possible hyperparameter combinations to identify the best set.

**Outcome :** The best combination of hyperparameters (grid_search.best_params_) was selected.

The model was trained with the best parameters found from the GridSearchCV.

## Model Evaluation

As a prerequisite threshold tuning was done in order to find the optimal threshold to maximize churn detection.

- Default threshold of 0.5 may not be optimal for imbalanced data.
- A range of thresholds from 0.1 to 0.7 (step 0.01) was tested to maximize F1-score on the test set.
- The threshold giving the best F1-score was selected for final predictions which is 0.4299.

**1. Overfitting and Underfitting Detection**

Due to lack of data, there was no split for validation to check for underfitting or overfitting so the below techniques was used:

- Overfitting Check: The difference between train and test metrics was examined. A significant gap (e.g., >0.1 in accuracy or ROC-AUC) would indicate overfitting.
- Underfitting Check: Low performance on both sets (ROC-AUC <0.65) indicates the model lacks complexity and fails to model the relationships within the data.

The ROC-AUC and accuracy gap between train and test was minimal, suggesting no significant overfitting. Both training and test metrics were sufficiently high, indicating the model captured the relevant patterns without underfitting or overfitting.

## 2. Feature Importance

Based on the plot contract and monthly charges have influenced the decision tree the most.

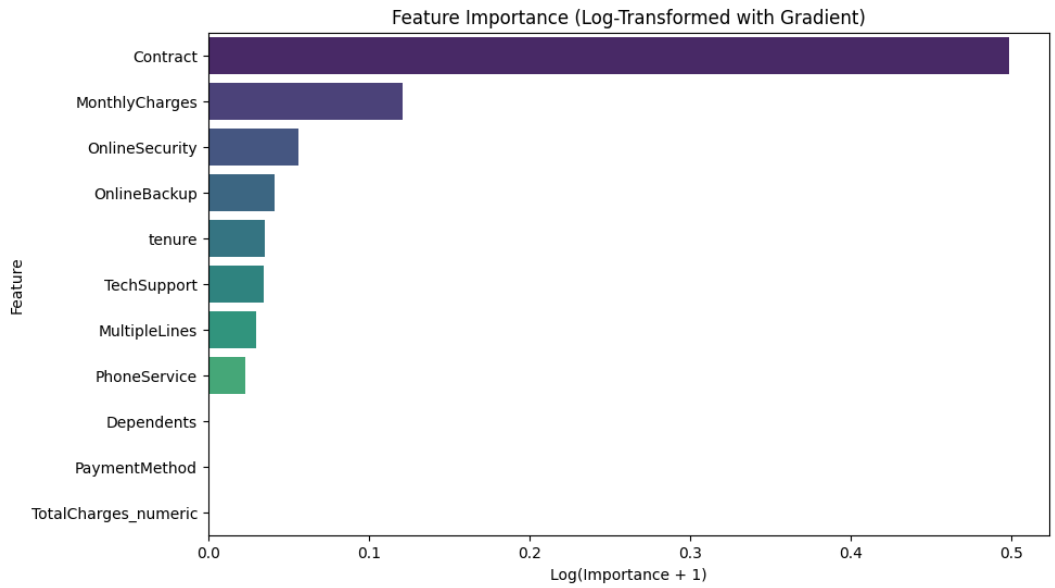



*Figure 2 - Feature Importance (after feature reduction)*

## 3. Cross-Validation

The training data underwent 5-fold stratified cross-validation to ensure consistent model performance.

- F1-scores for each fold: [0.81045752 0.81864696 0.81332567 0.81103286 0.82087782]
- Mean F1-score (CV): 0.8148681661814727

This shows that the perform is stable among various data groups of the training data indicating no signs of overfitting. The mean value shows that when the model is used to test data similar to the training set, we can achieve an F1-score of 0.815.

## 4. Evaluation Metrics (Train vs Test)

| | Accuracy | Precision | Recall | F1 Score | AUC Score |
|---|---|---|---|---|---|
| Train Set | 0.7876 | 0.7669 | 0.8265 | 0.7956 | 0.8668 |
| Test Set | 0.7480 | 0.5179 | 0.7353 | 0.6077 | 0.8167 |

**Interpretation**

- The training scores are higher, as expected after pruning and SMOTE oversampling.
- The model still generalizes well, with strong recall on the test set, which is crucial for churn prediction.
- The drop in precision on the test set is typical when SMOTE is used, as oversampling increases false positives**.**

**5. Classification Report (Test)**

| | Precision | Recall | F1-Score | Support |
|---|---|---|---|---|
| 0 (Non-Churn) | 0.8872 | 0.7527 | 0.8144 | 1035 |
| 1 (Churn) | 0.5179 | 0.7353 | 0.6077 | 374 |
| Accuracy | | | 0.7480 | 1409 |
| Macro Avg | 0.7026 | 0.7440 | 0.7111 | 1409 |
| Weighted Avg | 0.7892 | 0.7480 | 0.7596 | 1409 |

The model performs better on the non-churn class than on churn class. Precision and F1 score are high, indicating accurate identification of majority class customers, the churn class lower precision and moderate F1 score, showing that predictions for churners are less precise. Recall is comparable, suggesting that the model captures most actual churners, but also misclassifies a huge number of non-churners as churners. This difference is primarily due to class imbalance in the test set, as SMOTE was applied only during training and not on the test data.

**5. Confusion Matrix Visualization**

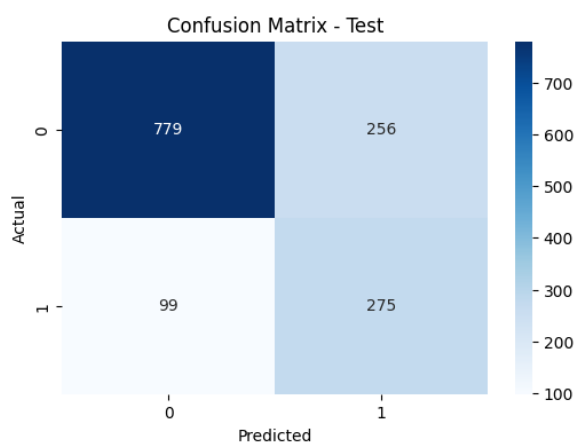



*Figure 3 - Confusion Matrix (Decision Tree)*

1. True Negatives (779)
2. True Positives (275)
3. False Positives (256)
4. False Negatives (99)

This demonstrates that the model can successfully recognize churners (high recall), it slightly over-predicts churn for some non-churn customers, which is acceptable from a business perspective

where identifying potential churners is more critical than occasionally misclassifying loyal customers.

### 6. ROC Curve

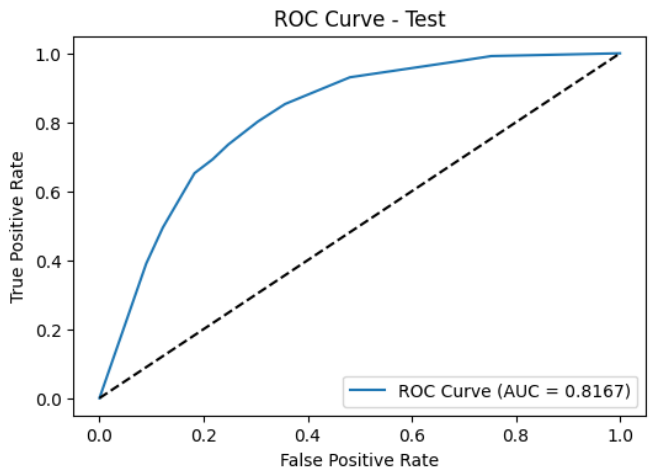



*Figure 4 - ROC Curve (Decision Tree)*

The curve highlights how well the model can differentiate between customers who churn and those who stay. The curve rises sharply above the diagonal, demonstrating strong predictive performance. With an AUC of 0.8167, the model shows solid discriminative ability, meaning there is about an 81.7% probability that it will assign a higher score to a randomly selected churner than to a non-churner. This high AUC reflects the effect of SMOTE in improving minority class detection, rather than overfitting, and confirms that the model reliably separates the two classes.

### 7. Precision-Recall Curve

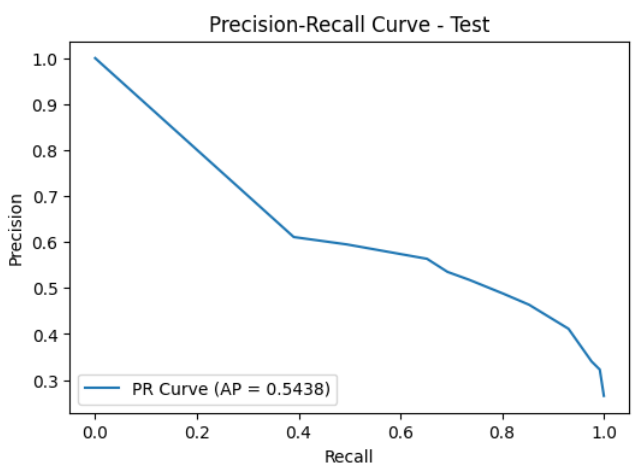
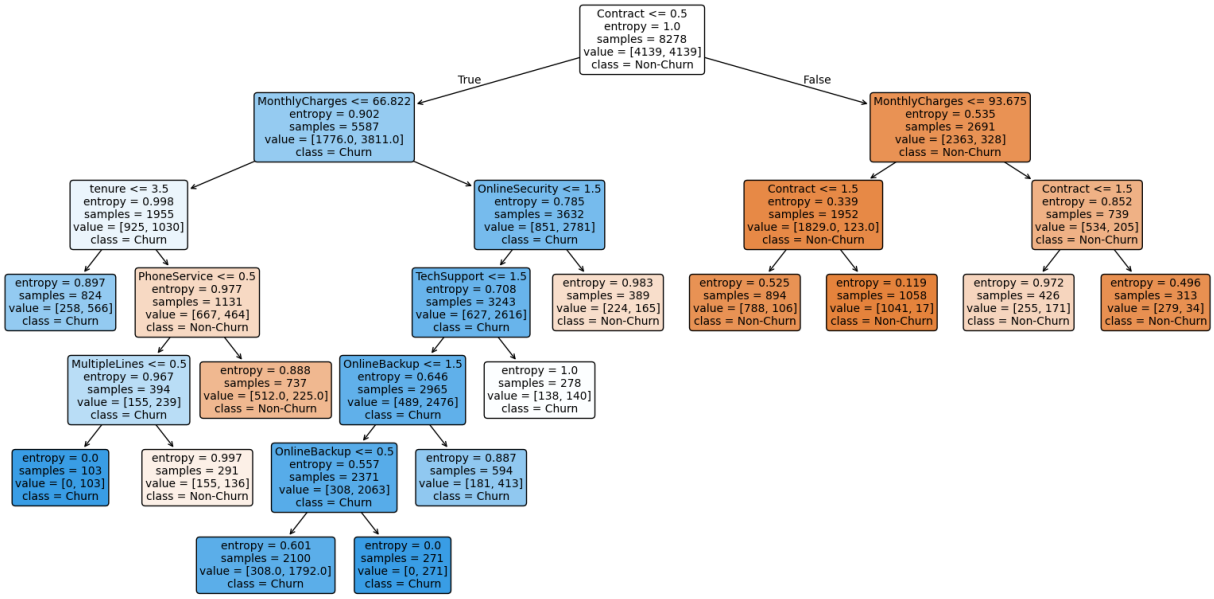



*Figure 5 - Precision-Recall Curve (Decision Tree)*

Shows the balance between Precision and Recall. It achieves perfect Precision (1.0) at very low Recall, meaning that when the model is highly selective, its few predictions are correct. When the model tries to capture more positive cases, its precision drops significantly.

# 8. Decision Tree Visualization